Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.М71-мм

Встраиваемый предметно-ориентированный язык для описания правил редукции вычислителя на основе Interaction Nets

Кубышкин Ефим Алексеевич

Отчёт по производственной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н., Григорьев С. В.

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **4**

2. Обзор **5**

2.1. Правила редукции Interaction Nets 5

2.2. Интерфейс правил в ядре вычислителя 5

2.3. Старая версия правил 7

2.4. Существующие реализации и встраивание языка 8

3. Требования и проектирование **10**

3.1. Требования 10

3.2. Подход к реализации 10

4. Реализация **12**

4.1. Объявление набора агентов 12

4.2. Сборщик правила 13

4.3. Тип правила и таблица правил 14

5. Апробация **15**

5.1. Правила для λ-исчисления 15

5.2. Тесты 16

5.3. Оценка упрощения 17

Заключение **19**

Список литературы **20**

Введение

Современные вычислительные системы сталкиваются с растущей потребностью в параллельной обработке данных [8]. Однако традиционные архитектуры процессоров часто неэффективны при нерегулярном параллелизме, например при обработке графов [2]. Один из подходов к таким задачам — крупнозернистые реконфигурируемые архитектуры (CGRA) [11], сочетающие гибкость программирования с производительностью специализированной аппаратуры.

Нерегулярный параллелизм естественно выражается в Interaction Nets [7] — модели вычислений, в которой граф из агентов переписывается локальными правилами. В 2024 году начат проект Lamagraph[1] — вычислитель на основе Interaction Nets, написанный на Clash [1] — HDL на основе Haskell.

Ядро вычислителя — управляющий автомат, модуль редукции и менеджер памяти — параметризуется набором агентов и правилами редукции. До текущей работы правила записывались вручную на Clash: разработчик сам распределял адреса памяти по новым агентам, раскладывал соединения по векторам фиксированной длины, дополнял их пустыми значениями и следил, чтобы число адресов совпадало с числом новых агентов. Такой код привязан к числу портов и к порядку агентов в паре; добавление вида агента может потребовать правки уже написанных правил, а часть ошибок проявляется лишь при симуляции.

В работе разработан встраиваемый предметно-ориентированный язык (EDSL) для описания правил редукции вычислителя на основе Interaction Nets: правило записывается как сеть, заменяющая пару взаимодействующих агентов, без адресов и структур памяти ядра, и образует экземпляр того же интерфейса, что и старая версия правил, без изменения ядра. Правила на EDSL сверяются в тестах со старой версией правил; упрощение оценено по объёму правил, местам правки и этапу обнаружения ошибок.

[1] GitHub проекта: https://github.com/Lamagraph/interaction-nets-in-fpga (дата обращения: 1 ноября 2026 г.).

1. Постановка задачи

Целью работы является упрощение описания правил редукции для вычислителя на основе Interaction Nets с помощью EDSL. Для её выполнения были поставлены следующие задачи.

1. Выполнить обзор правил редукции Interaction Nets и интерфейса правил ядра, проанализировать старую версию правил, сформулировать требования к EDSL (разделы 2 и 3).

2. Спроектировать и реализовать EDSL, правила на котором подключаются к ядру вычислителя без изменения его кода (разделы 3 и 4).

3. Проверить правила на EDSL тестами и e2e-тестированием (end-to-end) — редукцией сети в симуляции ядра целиком (подраздел 5.2).

4. Сравнить старую версию правил и правила на EDSL по объёму, числу мест правки и этапу обнаружения ошибок (подраздел 5.3).

2. Обзор

В обзоре вводятся понятия, разбираются недостатки старой версии правил, описывает образец записи правил и обосновывается выбор встраиваемого языка.

2.1. Правила редукции Interaction Nets

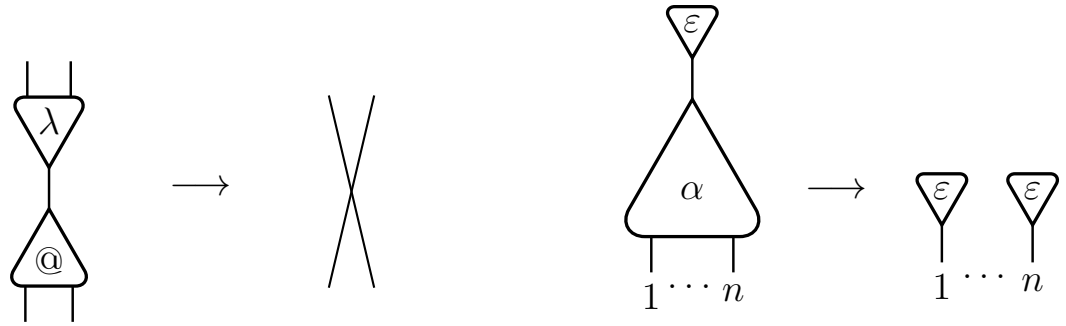
Interaction Nets [7] состоит из агентов с попарно соединёнными портами. Агент помечен видом α — символом алфавита Σ — и имеет один главный порт и $Ar(\alpha)$ дополнительных; арность $Ar(\alpha)$ по определению — число дополнительных портов агентов α. Два агента, соединённые главными портами, образуют активную пару. Правило редукции $\alpha \bowtie \gamma$ заменяет её новой сетью с теми же внешними портами — портами вне пары, к которым были подключены её дополнительные порты; код этой сети — тело правила. Порядок агентов в паре — ориентация пары; правило, записанное один раз для обеих ориентаций, симметрично. Рис. 1 иллюстрирует правила β-редукции (а) для агентов применения (@) и абстракции (λ), где внешние порты соединяются попарно, и стирания (б), где к n дополнительным портам агента α подключаются новые агенты стирания ε.

2.2. Интерфейс правил в ядре вычислителя

Синтезируемый код на Clash не может использовать списки и неограниченную рекурсию, а данные представляются векторами фиксированной длины `Vec n a`. Ядро вычислителя, класс `INet` и старая версия правил — правила на Clash без EDSL — описаны в [13].

Как показано на рис. 2, ядро получает правила только через экземпляр класса типов `INet`; экземпляр на EDSL имеет тот же тип, что и экземпляр старой версии.

Класс `INet`, показанный на листинге 1, параметризован типом метки `agentType`, хранящей вид агента в `Node`, и верхними границами `nodesNumber`, `edgesNumber`, `portsNumber`: наибольшим числом новых



а) б)

Рис. 1: Правила редукции: а) β-редукция; б) стирание

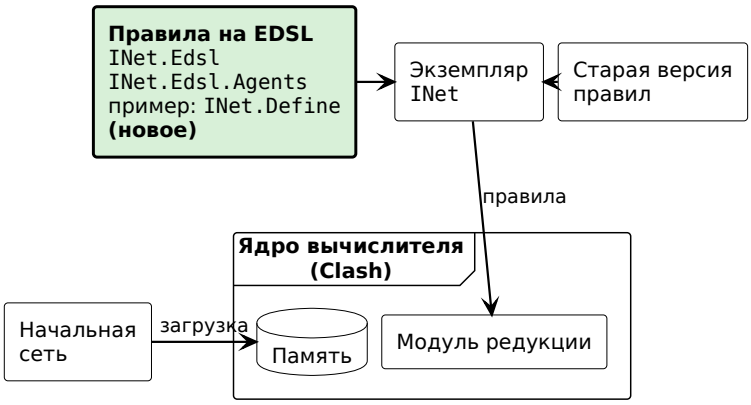


Рис. 2: Место правил редукции в стеке LAMAGRAPH: ядро получает правила через экземпляр `INet`; зелёным — разработанное в этой работе

узлов и рёбер и числом дополнительных портов узла.

Листинг 1: Класс `INet` — интерфейс, через который ядро получает правила (сокращено)

```
class INet agentType nodesNumber edgesNumber portsNumber
  where
  getAddressesAllocationCount ::
    agentType -> agentType -> Index CellsNumber
  makeReduction :: Vec nodesNumber (Maybe AddressNumber)
    -> LoadedNode portsNumber agentType -- левый узел пары
    -> LoadedNode portsNumber agentType -- правый узел пары
    -> ReduceRuleResult ...
```

Функция класса `getAddressesAllocationCount` возвращает число выделяемых адресов — адресов, отводимых паре менеджером памяти.

Функция класса `makeReduction` получает пул свободных адресов, выделенных менеджером памяти, и узлы пары, выбирает тело правила по паре видов агентов (диспетчеризация) и возвращает векторы новых рёбер и узлов, дополненные пустыми значениями при необходимости. Экземпляр корректен, только если число выделяемых адресов равно числу адресов, которые `makeReduction` берёт из пула, а для пары без правила обе функции возвращают `errorX` — неопределённое значение Clash, вызывающее ошибку при симуляции; типы функций этого не выражают, и проверить это можно только тестами.

2.3. Старая версия правил

На листинге 2 показано старая версия правила стирания из модуля `Core.Concrete.ReduceRulesLambda`; комментарии `(1)`–`(3)` отмечают его недостатки.

Листинг 2: Правило стирания старой версии для агента применения или абстракции, (1)–(3) — недостатки (сокращено)

```
eraseToAbstractOrApplyRule ::
  Vec 2 (Maybe AddressNumber) -> LoadedNode 2 ... ->
  LoadedNode 2 ... -> ReduceRuleResult 2 2 2 ...
eraseToAbstractOrApplyRule
  ((Just address1) :> (Just address2) :> Nil)
  _ loadedNodeAny = ReduceRuleResult def $ zipWith -- (1)
    (\address -> Just . maybe (error "...") -- (2)
      (\connection -> LoadedNode
        (Node connection def Erase) address)) -- (3)
    (address1 :> address2 :> Nil)
    (loadedNodeAny ^. containedNode . secondaryPorts)
eraseToAbstractOrApplyRule _ _ _ = error "..."
```

(1) Правило несимметрично: образец `_` отбрасывает узел стирания, обратную ориентацию обрабатывают другие ветви диспетчеризации.

(2) Если слот партнёра — позиция дополнительного порта в векторе портов узла — пуст (`Nothing`), вызывается `error`.

(3) Тело строит узлы из структур ядра `Node` и `LoadedNode` (узел с адресом).

Из-за (1) растёт число мест правки при добавлении вида агента, из-за (2) ошибки проявляются во время выполнения; сравнение — в подразделе 5.3.

2.4. Существующие реализации и встраивание языка

Ниже рассмотрено, как задают агенты и правила три реализации Interaction Nets и Inpla, взятый за образец записи правил.

Inpla [3] — язык программирования на Interaction Nets с текстовой записью правил и программной реализацией. Агент записывается именем с переменными на месте дополнительных портов, а правило — активной парой слева от `=>` и списком соединений `~` справа: правило стирания — `Erase >< Apply(a, b) => Erase~a, Erase~b;`.

graph-rewriting [9] — библиотека переписывания графов на Haskell. Агенты — конструкторы типа данных с портами в полях записи; список портов библиотеке передаёт вручную написанный экземпляр класса. Правило — значение монады образцов: образец `activePair` находит активную пару в любой ориентации, а правая часть строится из новых агентов (`byNode`), новых рёбер (`byEdge`) и соединений (`byWire`).

HINet [6] — параллельный интерпретатор текстового языка Inets на Haskell. Правило в Inets записывается, как в Inpla: например, сложение нуля — `Add(r,y) >< Z() => r~y;`.

Кодирование Interaction Nets в OCaml [5] записывает сеть обычным кодом: агент — конструктор типа `agent`, а его аргументы — то, что подключено к дополнительным портам. Тип агента помечен как `pos` или `neg`, и активная пара всегда состоит из агентов `pos` и `neg`. Как показано на листинге 3, все правила — ветви одной функции `apply_rule` от такой пары, а правая часть создаёт новые активные пары оператором `-><-`: число `Int n`, встретив `Add (r, b)`, образует пару из `b` и `AddAux (n, r)`.

Листинг 3: Правила сложения в кодировании Interaction Nets в OCaml (пример из [5], сокращено)

```
let rec apply_rule : type a.
    (a, pos) agent -> (a, neg) agent -> unit =
  fun a1 a2 -> match a1, a2 with
  | Int n, Add (r, b) -> b ><- AddAux (n, r)
  | Int m, AddAux (n, r) -> Int (n + m) ><- r
  | ...
```

Выбор встраиваемого языка. Все рассмотренные реализации устроены по одной схеме: правая часть правила создаёт новые агенты и соединяет порты, и для записи правила достаточно этих двух действий. Но они хранят сеть в динамических структурах и не порождают описания аппаратуры. Поэтому выбран язык, встраиваемый в Haskell: правило на нём компилируется вместе с ядром и проверяется системой типов, а отдельному текстовому языку нужны синтаксический анализатор и генератор кода.

3. Требования и проектирование

Требования к EDSL выводятся из недостатков старой версии правил, разобранных в подразделе 2.3; из требований следует подход к реализации, детали которой описаны в разделе 4.

3.1. Требования

- В теле правила нет адресов и структур памяти ядра: тело описывает только сеть, заменяющую активную пару.
- Вид агента и его арность объявляются в одном месте: остальные представления вида получаются из объявления.
- Число портов агента в теле следует из объявленной арности: несоответствие арности обнаруживается при компиляции.
- Число адресов задаётся вместе с правилом: число выделяемых адресов не записывается отдельно от тела.
- Одно правило задаёт обе ориентации пары: обратная ориентация не записывается отдельно.

3.2. Подход к реализации

Язык встраивается в Haskell (подраздел 2.4): тело правила — код Clash, компилируемый вместе с ядром без промежуточного интерпретатора.

Вид агента и его арность объявляются один раз, а тип агента и его арность на уровне типов порождаются из объявления при компиляции средствами Template Haskell [10]. Так арность становится доступна проверке типов: тело правила обходит порты партнёра, не перечисляя их, а несоответствие арности обнаруживает компилятор.

Тело правила описывает сеть, заменяющую пару, двумя действиями: создать агент и соединить два любых конца — свободный (неподключённый), порт нового агента или внешний порт. Адреса новым агентам

язык назначает сам, а представление агентов в памяти ядра строит после выполнения тела, поэтому в теле правила нет явной работы ни с тем, ни с другим.

Размеры векторов результата задаются одной верхней границей на весь набор агентов, а её превышение (переполнение) обнаруживается во время выполнения. Число новых агентов правила объявляется в его типе, и число выделяемых адресов берётся оттуда.

Правила задаются списком, где каждая неупорядоченная пара видов агентов указана один раз; таблица правил, правило для обратной ориентации и экземпляр `INet` порождаются из него автоматически. Пропущенная пара, как и в старой версии правил, обнаруживается во время выполнения.

4. Реализация

EDSL, реализующий подход из подраздела 3.2, состоит из трёх модулей, не зависящих от набора агентов:

1. `INet.Edsl` — сборщик правила (монада тела правила), примитивы `fresh` и `wire` и тип `Rule`;
2. `INet.Edsl.Agents` — генератор объявлений `declareAgents`;
3. `INet.Edsl.Dispatch` — генератор таблицы правил `declareINet`.

Как показано на рис. 3, в модуле `INet.Define` для набора {`Apply`, `Abstract`, `Erase`, `Duplicate`} вручную пишутся объявление набора, правила и список правил, а остальное порождается.

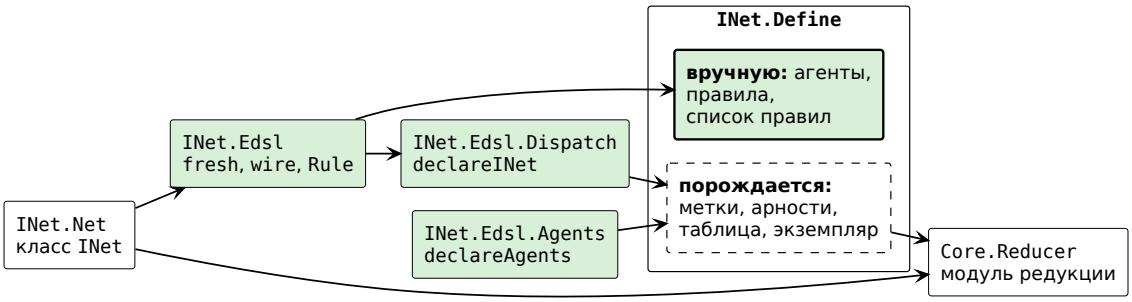


Рис. 3: Устройство EDSL: модули (зелёные — разработаны в этой работе), их зависимости (стрелка — от модуля к использующему его) и путь правила до ядра вычислителя

4.1. Объявление набора агентов

На листинге 4 показано объявление набора агентов в вызове генератора `declareAgents`.

Генератор проверяет список при компиляции (имена, арности) и порождает:

- перечисление меток `AgentSimpleLambda` (`ApplyLabel`, ...) для хранения вида агента в `Node`;
- семейство типов `AgentArity` — функцию на уровне типов от вида к арности;
- синглтон `Agent`, тип значения которого однозначно определяет вид агента;

Листинг 4: Объявление набора агентов λ-исчисления: вид агента — одна строка «имя, арность»

```
$( declareAgents
     "AgentSimpleLambda"
     [ ("Apply", 2)
     , ("Abstract", 2)
     , ("Erase", 0)
     , ("Duplicate", 2)
     ]
 )
```

- по экземпляру класса `AgentLabel` на вид, связывающему синглтон (например, `Apply`) с меткой `ApplyLabel` и арностью `AgentArityOf`.

Синглтон, семейство типов и класс нужны, чтобы над арностью можно было выполнять арифметику на уровне типов: с её помощью вычисляется число новых агентов после применения правила, например, дубликатор с агентом `Agent k` порождает `2 + AgentArityOf (Agent k)` новых агентов.

4.2. Сборщик правила

На листинге 5 приведены сигнатуры основных средств EDSL: ссылки на порт, сборщика правила и двух основных примитивов — `fresh` и `wire`. Ссылка на порт `PortPointer` указывает на свободный конец, главный или дополнительный порт. Параметры `edgesNumber`, `nodesNumber`, `portsNumber` сборщика правила `RuleBuilder` — верхние границы экземпляра, `agentType` — тип метки.

Функция `fresh` берёт из пула очередной адрес и создаёт накопитель соединений нового агента. Функция `wire` различает свободный конец, порт нового агента и внешний порт и действует так:

- свободный конец и порт нового агента — порт помечается неподключённым;
- внешний порт и внешний или свободный конец — добавляется

Листинг 5: Ссылка на порт, сборщик правила и примитивы (сигнатуры, сокращено)

```
data PortPointer
  = Free | Main AddressNumber | Pointer AddressNumber Int

type RuleBuilder edgesNumber nodesNumber portsNumber
  agentType = State (RuleState edgesNumber ...)

fresh :: (AgentLabel agentSingleton agentType, ...)
  => agentSingleton -> RuleBuilder ... (AddressNumber,
       Vec (AgentArityOf agentSingleton) PortPointer)
wire :: PortPointer -> PortPointer -> RuleBuilder ... ()
```

ребро;

- порт нового агента и внешний порт или порт нового агента — соединение записывается в накопители.

Функция `runReduce` выполняет тело и превращает накопители в `Node`; результат имеет тип результата `makeReduction`, а переполнение и исчерпание пула дают `errorX`.

4.3. Тип правила и таблица правил

Правило на EDSL — значение типа `Rule`, содержащее функцию от двух узлов пары, которая возвращает тело; число новых агентов `freshNodesNumber` задаётся в типе правила, и функция `ruleAllocationCount` возвращает его как число выделяемых адресов. Тип `SomeRule` скрывает это число, чтобы разные правила помещались в одну таблицу. Список правил генератор `declareINet` превращает при компиляции в таблицу `canonicalRule` (отсутствие правила — `Nothing`; ориентация агентов неважна), функцию `ruleFor` и экземпляр `INet`; пустой список и повтор пары дают ошибку компиляции. Функция `ruleFor`, не найдя пару, ищет обратную ориентацию и применяет `mirrorSomeRule`, выполняющую тело с переставленными узлами; для пары без правила экземпляр возвращает `errorX`.

5. Апробация

На EDSL записаны правила набора агентов λ-исчисления; они проверены unit и интеграционными тестами, и на них оценено упрощение.

5.1. Правила для λ-исчисления

Для заданного набора агентов `{Apply, Abstract, Erase, Duplicate}` и правил порождается экземпляр `INet AgentSimpleLambda 4 2 2`: правило дублирования создаёт до четырёх новых агентов, остальные правила — не больше двух. На листинге 6 показано правило β-редукции с рис. 1 (а). Тело `betaBody` получает ссылки на внешние порты агентов применения и абстракции и соединяет их попарно функцией `wire`. Новых агентов правило не создаёт, поэтому первый параметр типа `Rule` — число новых агентов — равен нулю; функция `ruleFromSecondaries` передаёт телу ссылки на порты обоих узлов пары.

Листинг 6: Правило β-редукции на EDSL: внешние порты соединяются попарно, новых агентов нет

```
betaBody :: Vec 2 PortPointer -> Vec 2 PortPointer
         -> RuleBuilder 2 4 2 AgentSimpleLambda ()
betaBody leftPorts rightPorts = do
  wire (leftPorts !! sec2Port0) (rightPorts !! sec2Port0)
  wire (leftPorts !! sec2Port1) (rightPorts !! sec2Port1)

betaRule :: Rule 0 2 4 2 AgentSimpleLambda
betaRule = ruleFromSecondaries betaBody
```

На листинге 7 показано правило стирания на EDSL; оно состоит из двух функций, отмеченных комментариями `(1)` и `(3)`.

(1) Тело `eraseAnyBody` получает вектор ссылок на внешние порты партнёра любой длины и возвращает сборщик правила.

Листинг 7: Правило стирания на EDSL: тело для портов партнёра любой арности и правило, связывающее его с узлами пары (скетч)

```
eraseAnyBody :: Vec partnerArity PortPointer      -- (1)
             -> RuleBuilder 2 4 2 AgentSimpleLambda ()
eraseAnyBody ports = mapPorts ports $ \port -> do -- (2)
  (addr, _) <- fresh Erase
  wire port (Main addr)

eraseRule :: (AgentArityOf (Agent k) ~ 2, ...)
  => Agent k -> Rule (AgentArityOf (Agent k)) ...  -- (3)
eraseRule _ = ruleFromSecondaries $ \_ rightPorts ->
  eraseAnyBody rightPorts
```

(2) Функция `mapPorts` из `INet.Edsl` обходит вектор ссылок без списков и рекурсии; для каждого порта создаётся агент `Erase`, и порт соединяется с его главным портом.

(3) Правило `eraseRule` получает синглтон вида партнёра и возвращает `Rule`; из ссылок на порты узлов пары, которые передаёт `ruleFromSecondaries`, тело стирания берёт порты правого. Число новых агентов в типе равно арности партнёра.

Тело правила дублирования `dupAnyBody` создаёт две копии партнёра и для каждого его дополнительного порта — агент `Duplicate`. На листинге 8 показан список правил набора: каждая из семи неупорядоченных пар задаётся одной записью, а таблица правил и экземпляр порождаются генератором `declareINet`, описанным в подразделе 4.3.

5.2. Тесты

Правила на EDSL сверяются с эталоном — старой версией правил — на наборе {`Apply`, `Abstract`, `Erase`} тестами на Hedgehog [4] и Tasty [12]. Дифференцирующие тесты на основе свойств для семи упорядоченных пар, у которых есть правило старой версии, сравнива-

Листинг 8: Список правил λ-исчисления, из которого порождаются таблица правил и экземпляр `INet` (сокращено)

```
$( declareINet "AgentSimpleLambda" 4 2 2
     [ rule "Apply" "Abstract" [|betaRule|]
     , rule "Erase" "Erase" [|eraseEraseRule|]
     , rule "Erase" "Apply" [|eraseRule Apply|]
     , ...
     , rule "Duplicate" "Apply" [|duplicateRule Apply|]
     , ...
     ]
 )
```

ют результаты двух правил на 100 случайных подсетях из 4–8 узлов; модульные тесты на фиксированных сетях проверяют β-редукцию, стирание, дублирование, число выделяемых адресов и код, порождаемый `declareAgents` и `declareINet`. У `Duplicate` эталона нет, поэтому правила с ним проверены только модульными тестами. Для e2e-тестирования взята сеть терма $(\lambda x.\, x)(\lambda y.\, y)$: ядро с экземпляром на EDSL, как и с экземпляром старой версии, приходит к корню — оставшейся абстракции — по тому же адресу. Из 86 тестов пакета `lamagraph-core` к EDSL относятся 49 (7 на основе свойств, 40 модульных и 2 e2e-теста); все проходят.

5.3. Оценка упрощения

Сравниваются объём тела правила, число мест правки при добавлении вида агента и этап обнаружения ошибок — характеристики, на которые влияют недостатки старой версии правил из подраздела 2.3. Сводка приведена в табл. 1; строки кода считались непустые, без комментариев.

Тела правил на EDSL короче в 2,3–5,3 раза и не содержат структур ядра; тело дублирования на EDSL занимает 10 строк, в старой версии такого правила нет. Служебный код на EDSL — список правил (14 строк) и функции, задающие правила (22 строки), — длиннее диспетчеризации

Таблица 1: Старая версия правил и правила на EDSL (строки кода и места правки: меньше — лучше)

Показатель	Старая версия	EDSL
Тело β-редукции, строк	7	3
Тело стирания, строк	16	3
Служебный код, строк	21	36
Мест правки для `Duplicate`	10	6
Ошибка арности партнёра	выполнение	компиляция

и вычисления числа выделяемых адресов в старой версии.

Для добавления `Duplicate` на EDSL потребовались строка в `declareAgents`, тело, правило с числом новых агентов в типе и три записи списка правил; в старой версии правил — конструктор перечисления, правило с раскладкой четырёх адресов по узлам, три ветви вычисления числа выделяемых адресов и пять ветвей диспетчеризации. Рост верхней границы новых узлов с двух до четырёх на EDSL требует правки параметров `2 4 2` в типах (в `INet.Define` — 8 строк) и аргумента `declareINet`, а в старой версии правил изменил бы сигнатуры всех правил и тело стирания, рассчитанное на два адреса. Оценка ограничена одним набором из четырёх видов агентов; места правки в старой версии правил подсчитаны по коду, без фактического добавления.

По этапу обнаружения ошибок EDSL переносит на компиляцию четыре класса: неверное число портов агента (длину вектора ссылок задаёт `fresh`; обращение по индексу `!!` вне арности даёт ошибку во время выполнения), некорректное объявление набора (проверяет `declareAgents`), повтор пары в списке правил (проверяет `declareINet`) и арность партнёра, отличную от ожидаемой (проверяют правила). Несовпадение числа адресов с телом и отсутствие правила для пары, как и в старой версии правил, обнаруживаются во время выполнения (первое — также тестом).

Таким образом, выполнены все требования подраздела 3.1; EDSL упрощает тело правила, объявление вида агента и задание ориентаций, но не служебный код.

Заключение

В ходе работы были получены следующие результаты.

1. Выполнен обзор правил редукции Interaction Nets и интерфейса правил ядра, проанализирована старая версия правил, сформулированы требования.
2. Спроектирован и реализован EDSL (генератор объявлений, сборщик правила, генератор таблицы правил); на нём записан экземпляр `INet` для λ-исчисления.
3. Правила проверены дифференцирующими и модульными тестами и e2e-тестированием в симуляции ядра (49 тестов, все проходят).
4. Выполнено сравнение: тела правил на EDSL в 2,3–5,3 раза короче и без структур ядра, пара — одна запись списка правил, четыре класса ошибок обнаруживаются при компиляции; служебный код длиннее.

Ссылка на Pull Request: https://github.com/Lamagraph/interaction-nets-in-fpga/pull/96; имя коммитера — KubEF.

Список литературы

[1] CλaSH: Structural Descriptions of Synchronous Hardware Using Haskell / Christiaan Baaij, Matthijs Kooijman, Jan Kuper et al. — 2010. — 09. — P. 714–721.

[2] Besta Maciej, Stanojevic Dimitri, Licht Johannes De Fine et al. Graph Processing on FPGAs: Taxonomy, Survey, Challenges. — 2019. — 1903.06697.

[3] Hassan Abubakar, Mackie Ian, Sato Shinya. An Implementation Model for Interaction Nets // Electronic Proceedings in Theoretical Computer Science. — 2015. — may. — Vol. 183. — P. 66–80. — URL: http://dx.doi.org/10.4204/EPTCS.183.5.

[4] Hedgehog: Release with confidence. — https://hedgehog.qa/. — Дата обращения: 29.10.2026.

[5] Huber Nikolaus, Yi Wang. An Encoding of Interaction Nets in OCaml // Electronic Proceedings in Theoretical Computer Science. — 2025. — Vol. 417. — P. 1–16.

[6] Kahl Wolfram. HINet: Interaction Nets in Haskell. — https://www.cas.mcmaster.ca/~kahl/Haskell/HINet/. — 2015. — Дата обращения: 03.11.2026.

[7] Lafont Yves. Interaction nets // Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. — POPL '90. — New York, NY, USA : Association for Computing Machinery, 1989. — P. 95–108. — URL: https://doi.org/10.1145/96709.96718.

[8] Qing Wu Maksym Spiryagin Colin Cole, McSweeney Tim. Parallel computing in railway research // International Journal of Rail Transportation. — 2020. — Vol. 8, no. 2. — P. 111–134. — URL: https://doi.org/10.1080/23248378.2018.1553115.

[9] Rochel Jan. graph-rewriting: Monadic graph rewriting of hypergraphs with ports and multiedges. — https://hackage.haskell.org/package/graph-rewriting. — Дата обращения: 03.11.2026.

[10] Sheard Tim, Peyton Jones Simon. Template Meta-programming for Haskell // Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell. — Haskell '02. — Association for Computing Machinery, 2002. — P. 1–16. — URL: https://doi.org/10.1145/581690.581691.

[11] A Survey of Coarse-Grained Reconfigurable Architecture and Design: Taxonomy, Challenges, and Applications / Leibo Liu, Jianfeng Zhu, Zhaoshi Li et al. // ACM Comput. Surv. — 2019. — oct. — Vol. 52, no. 6. — 39 p. — URL: https://doi.org/10.1145/3357375.

[12] Tasty: Modern and extensible testing framework. — https://hackage.haskell.org/package/tasty. — Дата обращения: 03.11.2026.

[13] Кубышкин Ефим Алексеевич. Разработка прототипа специализированного процессора, основанного на Interaction Nets. — 2025.